

Documentation technique

ECF Vite et Gourmand - Lesaffre Xavier

Réflexions initiales technologiques

Pour les outils de conception, le choix s'est porté sur ceux déjà pratiqués en formation, Figma pour le maquettage, VSCode pour le développement, QuickDBD pour le diagramme relationnel, avec une expérience plus limitée sur draw.io, découvert principalement au cours de ce projet pour la réalisation des diagrammes UML et du MCD.

Pour le stack back-end, j'ai fait le choix de ne pas utiliser de framework et de rester sur du PHP natif avec PDO, pour manipuler directement les mécanismes de base du développement web (requêtes SQL préparées, gestion de session, structure MVC artisanale), afin d'éviter que ce soit des outils qui s'en charge en en cachant le fonctionnement et pour mieux consolider mes connaissances acquises lors de la formation. Le choix de passer par MySQL m'a semblé cohérent avec le sujet du projet, qui impose une base de données relationnelle, de plus cette technologie a déjà été pratiquée lors de ma formation.

Pour le choix de MongoDB, c'est entièrement dû à l'obligation de développer des composants d'accès à des données NoSQL présente dans l'énoncé, j'ai choisi de l'utiliser dans la partie de gestion des statistiques de ventes, car cette partie est la plus détachable du reste des données. Cette intégration a nécessité une recherche personnelle approfondie sur les pipelines d'agrégation MongoDB, notion non abordée lors de ma formation initiale. De la même manière que pour le back-end, pour la partie front-end, je suis resté sans framework, avec HTML, CSS et Javascript vanilla pour travailler sur ma compréhension de la manipulation du DOM, ainsi que les requêtes AJAX natives.

La seule exception a été pour le service PHPMailer, car ce service demande une plus grande sécurité et apporte une complexité trop importante au vu de mes connaissances.

Configuration de l'environnement de travail

- Back-end : PHP 8.2 / PDO / MySQL
- NoSQL : MongoDB (statistiques de vente)
- Front-end : HTML / CSS / JavaScript (vanilla, AJAX)
- Dépendances : PHPMailer (envoi de mails), mongodb/mongodb (via Composer)
- Outils : XAMPP (environnement local), Figma (maquettage), draw.io (diagrammes), QuickDBD (diagramme relationnel), VSCode (codage)

Analyses du projet

Diagrammes de cas d'utilisation

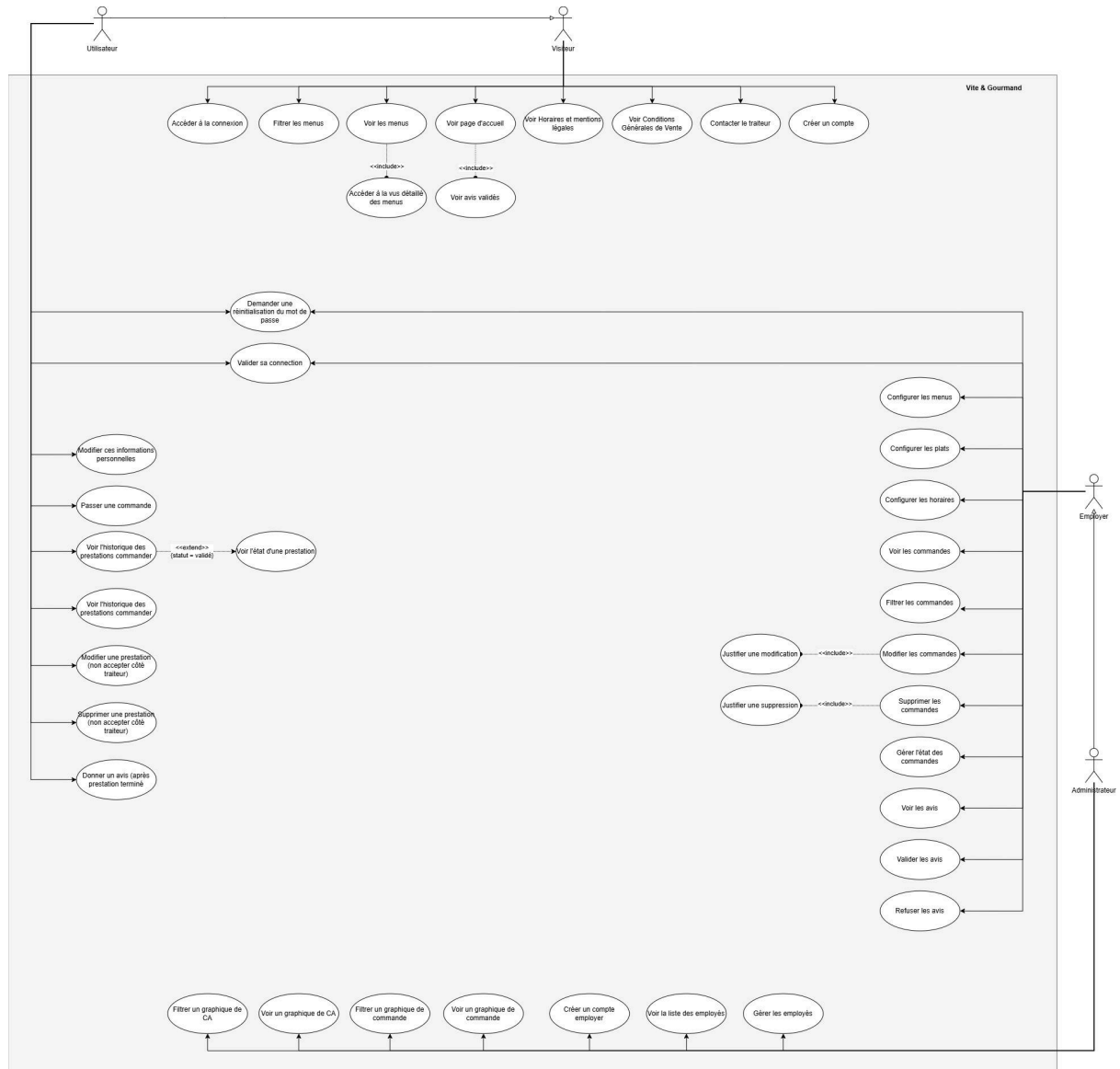


Diagramme de séquence

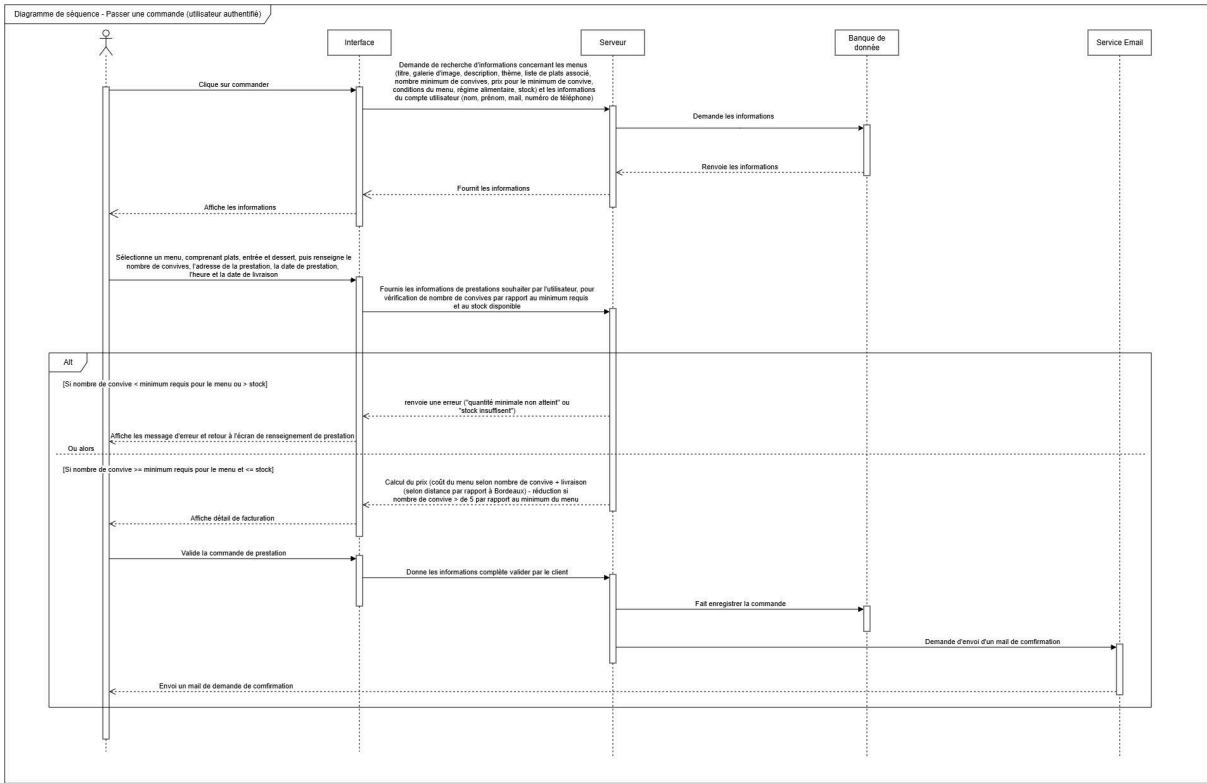


Diagramme de classes

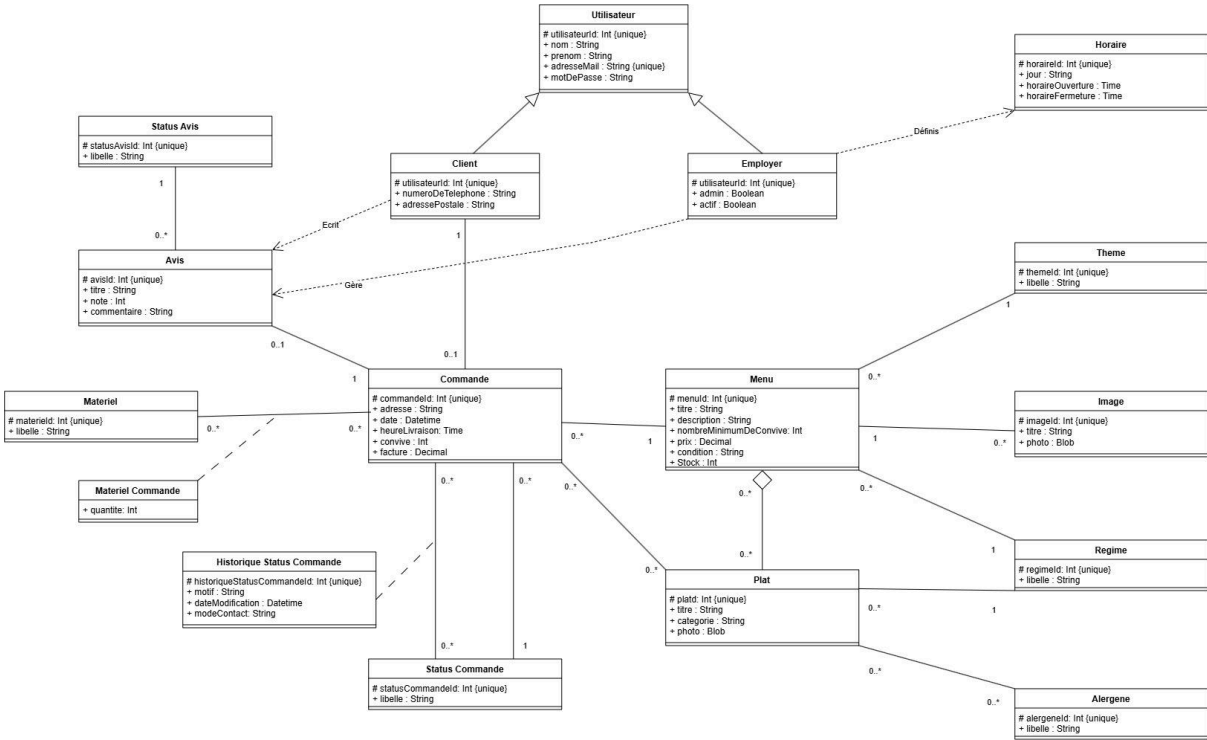
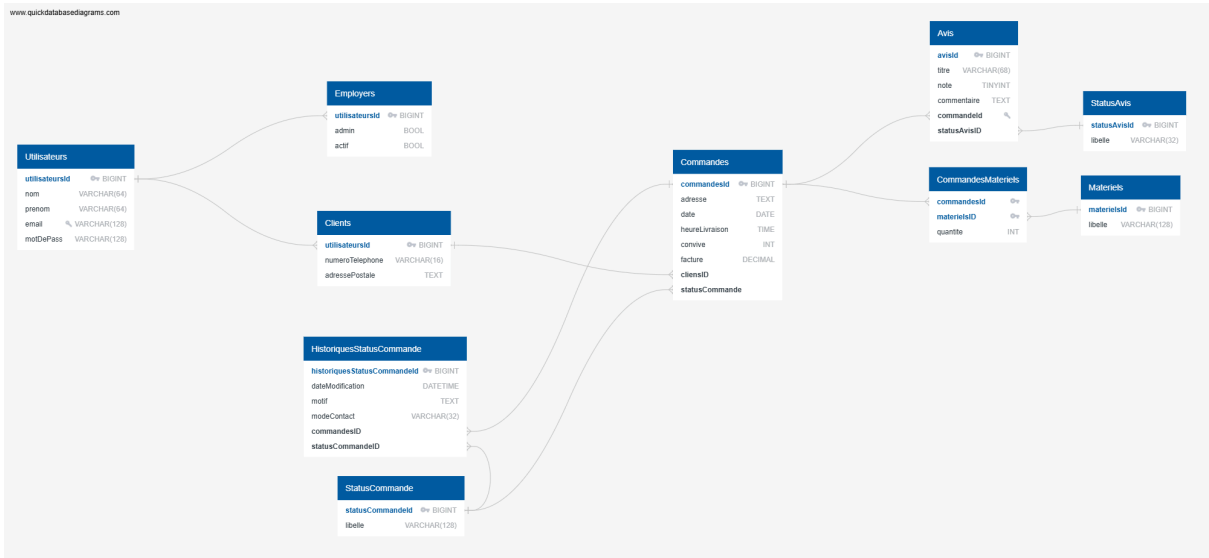
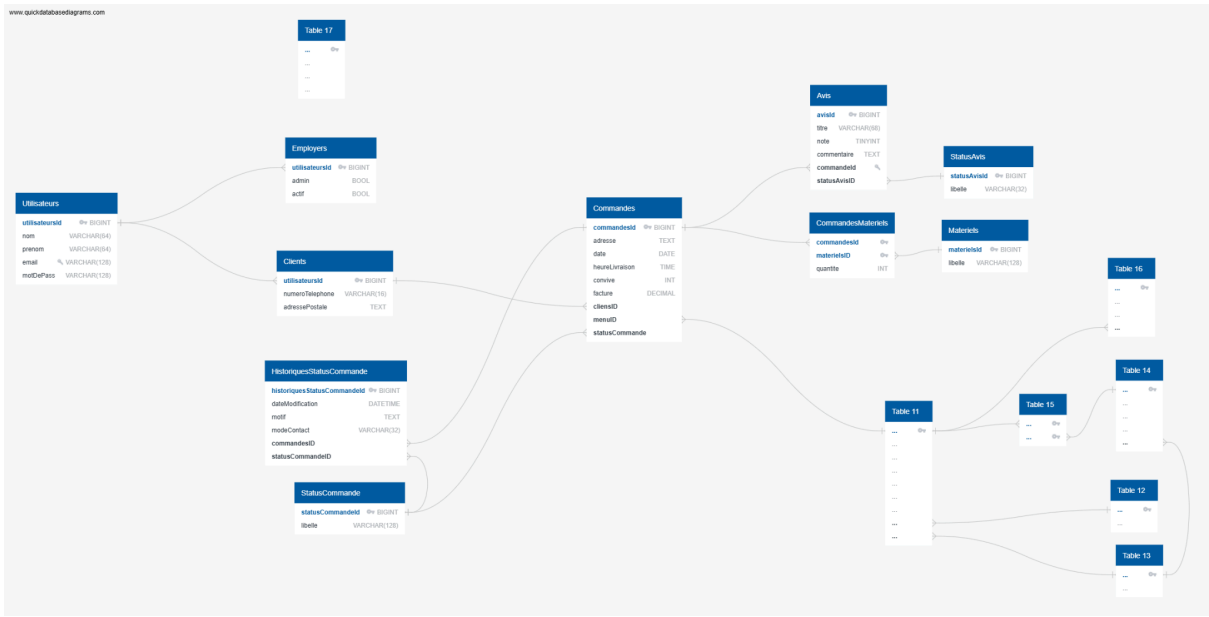
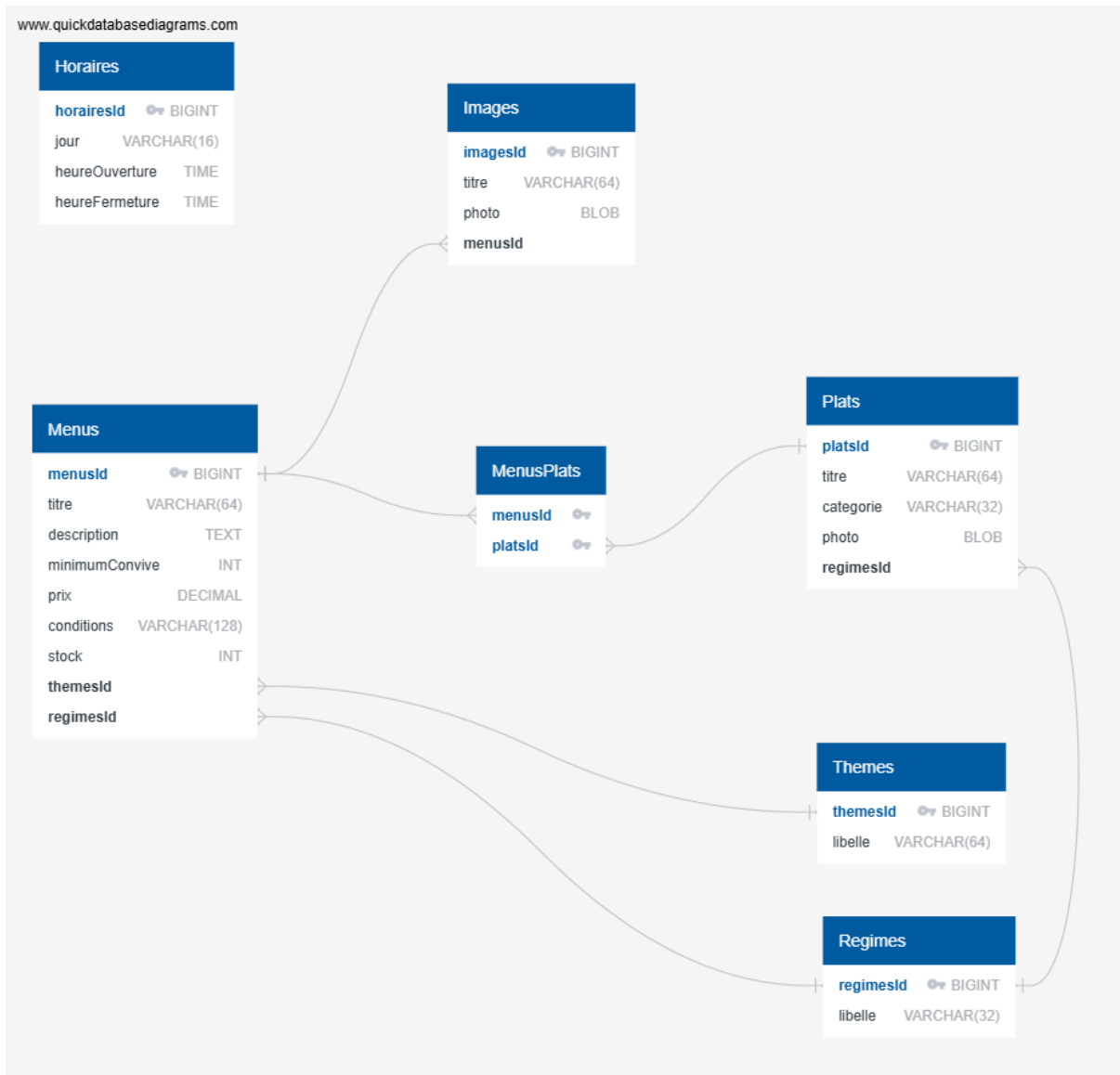


Diagramme relationnel





Déploiement

Le déploiement a été fait via la plateforme Heroku car celle-ci dispose d'un programme associé avec le GitHub Student Developer Pack qui donne accès à des crédits gratuits pour le déploiement d'un site. Une alternative avait été étudiée de passer par Render, un système gratuit, mais ce système de déploiement ne gère pas MySQL de manière native.

Avec Heroku, j'ai ajouté un add-on JawsDB pour la gestion de la partie base de données relationnelle MySQL.

Après avoir initialisé le déploiement, il a fallu effectuer une adaptation de la configuration pour que le fichier config pointe vers des informations de structure du déploiement avec Heroku et l'add-on JawsDB, avec une détection permettant de savoir si l'on est en ligne ou en local permettant de continuer à faire des changements en local.

J'ai également initié une base de données NoSQL via l'offre gratuite de la plateforme MongoDB Atlas, auquel les informations de connexion ont été ajoutées à Heroku et au fichier de configuration.

Une fois le tout prêt, j'ai push mon dossier git local vers le git distant Heroku, permettant ainsi le déploiement.

Le site n'a d'abord pas fonctionné pour plusieurs raisons, premièrement la base de données sous JawsDB était vide, ce faisant la page d'accueil qui réclame au minimum des données de statuts des avis pour fonctionner bloquait l'accès au site. Pour initialiser les données, j'ai dû ajouter Adminer à mon site pour effectuer la migration du schéma SQL vers JawsDB ainsi que des premières données fondamentales du site (Comptes administrateurs, Régimes alimentaires, Statuts d'avis, Statuts de commande) ainsi que des données n'ayant aucun système intégré dans le site pour les entrer (Matériels).

Une fois ces informations ajoutées, j'ai pu accéder au site et effectuer divers tests comme l'accès aux différentes pages, me connecter à un compte administrateur et visionner les onglets employé et administrateur, créer un compte client, sélectionner un menu et tenter de passer commande.

Ces tests m'ont permis de remarquer que deux dépendances d'extensions PHP étaient manquantes sur Heroku (mbstring et bcmath), il m'a fallu les ajouter au composer.json et mettre à jour le composer.lock.

Un autre problème est apparu avec la configuration de MongoDB Atlas qu'il m'a fallu régler directement sur le site de MongoDB Atlas. Ce problème était visible lorsque j'ai voulu finaliser complètement une commande pour voir la mise à jour des graphiques.

Notes réflexions technique

- Avis et commande

Vu que les avis dépendent des commandes, il devrait y avoir qu'un avis possible par commande effectuée par un client. Ce qui a amené à lier "Avis" à "Commande" dans le diagramme de classe.

- Etat de commande dans document NoSQL

J'ai considéré que pour le suivi des commandes dans les graphiques administrateur seul les commandes ayant atteint l'état "Terminée" seront ajoutées aux données de la base NoSQL, les chiffres reflèteront donc uniquement les commandes abouties soit le CA fait, pour ne pas risquer de voir apparaître des commandes pouvant encore être annulées ou modifiées (les commandes "en cours")

- E-mail et compte

Je sépare volontairement les comptes utilisateurs-clients et utilisateurs-employés, via des adresses mail distinctes, une privé et une professionnelle, permettant à un employé de se connecter soit en tant que client avec son mail privé, soit en tant qu'employé avec son mail pro

- Gestion des employés

La seule gestion que les administrateurs ont des employés dans l'énoncé semblant être la possibilité de rendre inactif un compte employé, j'ai estimé dans mon diagramme de cas d'utilisation que la gestion des employés serait le terme utilisé pour l'activation/désactivation des comptes employés

- Diagramme de séquence

Le cas développé dans le diagramme de séquence est celui d'un utilisateur connecté avec ses identifiants.

- Minimum convive et commande

Dans le HTML de la commande, une contrainte de minimum de convive pourrait être intégrée pour éviter aux utilisateurs de devoir vérifier cette contrainte en retournant sur le menu.

- Service mail et retour

A la fin d'une commande réussie, le mail est directement envoyé aux clients sans passer par un retour vers le serveur, ce faisant le serveur n'attend pas que le service mail ait fait son travail pour continuer.

- Rôle administrateur et table

Dans le diagramme de classe, être administrateur ne s'est pas traduit par une table fille d'employés, mais en une donnée booléenne définissant les droits d'accès des employés.

- Coût du menu

Pour simplifier les données et éviter de répondre à un besoin non mentionné dans l'énoncé, le prix et les conditions ne sont pas dupliqués entre le menu et les plats, il sera laissé au jugement de l'employé créant le menu de choisir des plats se raccordant au prix et aux conditions du menu. Mais dans un cadre réaliste ces données pourraient faire l'objet d'un débat avec le client pour savoir comment il compte réellement gérer le prix des menus par rapport aux prix des plats en eux même.

- Lien employé-avis dans UML

L'employé, bien que ne modifiant pas l'avis mais uniquement le statut de l'avis, a un lien direct avec la classe avis car c'est une classe entière et non un dérivé n'apportant qu'un attribut précis.

- Titre Avis

Dans l'énoncé aucun titre d'avis n'est mentionné, mais j'ai estimé que cela pouvait être un ajout à la visibilité des avis, permettant d'avoir deux visuel des avis, un juste avec une note et le titre (voir le menu concerné) et un autre plus complet en agrandissant le visuel.

- Titre image

Dans l'énoncé aucun titre d'image n'est demandé, mais dans le cadre d'un texte alternatif à l'image j'ai décidé que ce serait un titre qui serait affiché en remplacement, et que cette donnée serait à remplir pour la table de données.

- Switch dans le menu du site

Certains boutons du menu du site n'ayant plus de raison d'être après connexion, j'ai décidé que leur place serait utilisée pour d'autres actions, le bouton "connexion" devient donc "déconnexion" et "inscription" devient "mon espace" après connexion d'un utilisateur identifié. "Mon espace" redirigeant différemment en fonction du rôle de l'utilisateur (client, employé et administrateur).

- Adresse de prestation vs lieu

Dans l'énoncé il est fait autant mention d'une adresse de prestation, et d'un lieu dans le cadre de la livraison, dans mon wireframe j'ai considéré que le lieu de livraison serait une information principale car le livreur n'a pas forcément besoin de savoir où le repas aura réellement lieu si cela n'affecte pas la prestation, et j'ai juste ajouté un champ optionnel permettant de préciser des informations si une partie du matériel loué pour la prestation devait se faire dans un lieu distinct de la livraison

- Allergène commande

Etant donné que les allergènes sont liés aux plats, je fait apparaître leur mention uniquement dans la commande après que les plats concernés par des allergènes ont été sélectionnés.

- Mention légale et CGV

Dans un souci de place, les mentions légales et les conditions générales de vente sont assemblées dans la version mobile, mais séparées dans la version desktop qui dispose de plus de place.

- Galerie d'image

Deux approches différentes sont utilisées en fonction du support employé pour visiter le site, un carrousel horizontal pour la version mobile disposant de peu de place et étant une utilisation plus intuitive sur téléphone, alors qu'une liste verticale avec scrollbar est employée dans la version desktop bénéficiant de l'espace vertical de ce support.

- Visuel image internet

J'ai profité d'habiter proche de Bordeaux pour prendre une photo de la ville pour habiller le site, les autres visuels proviennent du site unsplash.com dans un souci de droit d'auteur.

- Notation Avis

Pour respecter le principe RGAA, les notes d'avis ont un texte alternatif noté sur cinq associé au bloc d'étoiles (exemple 4 sur 5)

- Frais de livraison

Le calcul des frais de livraison est simplifié à un forfait de 5€ hors Bordeaux (codes postaux 33000/33100/33200/33300/33800), sans variation kilométrique, en l'absence d'adresse de départ fixe du traiteur et pour rester dans le périmètre raisonnable du projet.

- Annulation de commande et historique statut

Pour ne pas obliger à fournir un motif et un mode de contact pour tous les changements de statut de commande, une contrainte conditionnelle en code a été appliquée plutôt qu'une contrainte de base de données.

- Limite de format d'image

Les images uploadées sont limitées au format PNG (validation par vérification du type MIME réel via finfo, pas seulement l'extension), pour simplifier la structure de stockage en base64.

- Adresse et informations complémentaires

L'adresse et les informations complémentaires de livraison sont fusionnées en une seule chaîne pour simplifier la structure de données, mais aussi car la différenciation des

informations a été oubliée en cours de développement et que sa correction aurait entraîné une chaîne de modification incompatible avec les délais du projet.

- Avis et gestion client

Un client peut retirer son avis à tout moment, y compris après validation, par cohérence avec sa liberté de s'exprimer ou de se rétracter

- Désactivation des plats

La désactivation d'un plat (soft delete) ne retire pas automatiquement ce plat des menus qui le contiennent déjà, l'employé doit penser à retirer manuellement le plat désactivé de chaque menu concerné, sans quoi il continuerait d'apparaître dans l'affichage public.